

Notebook for EDA

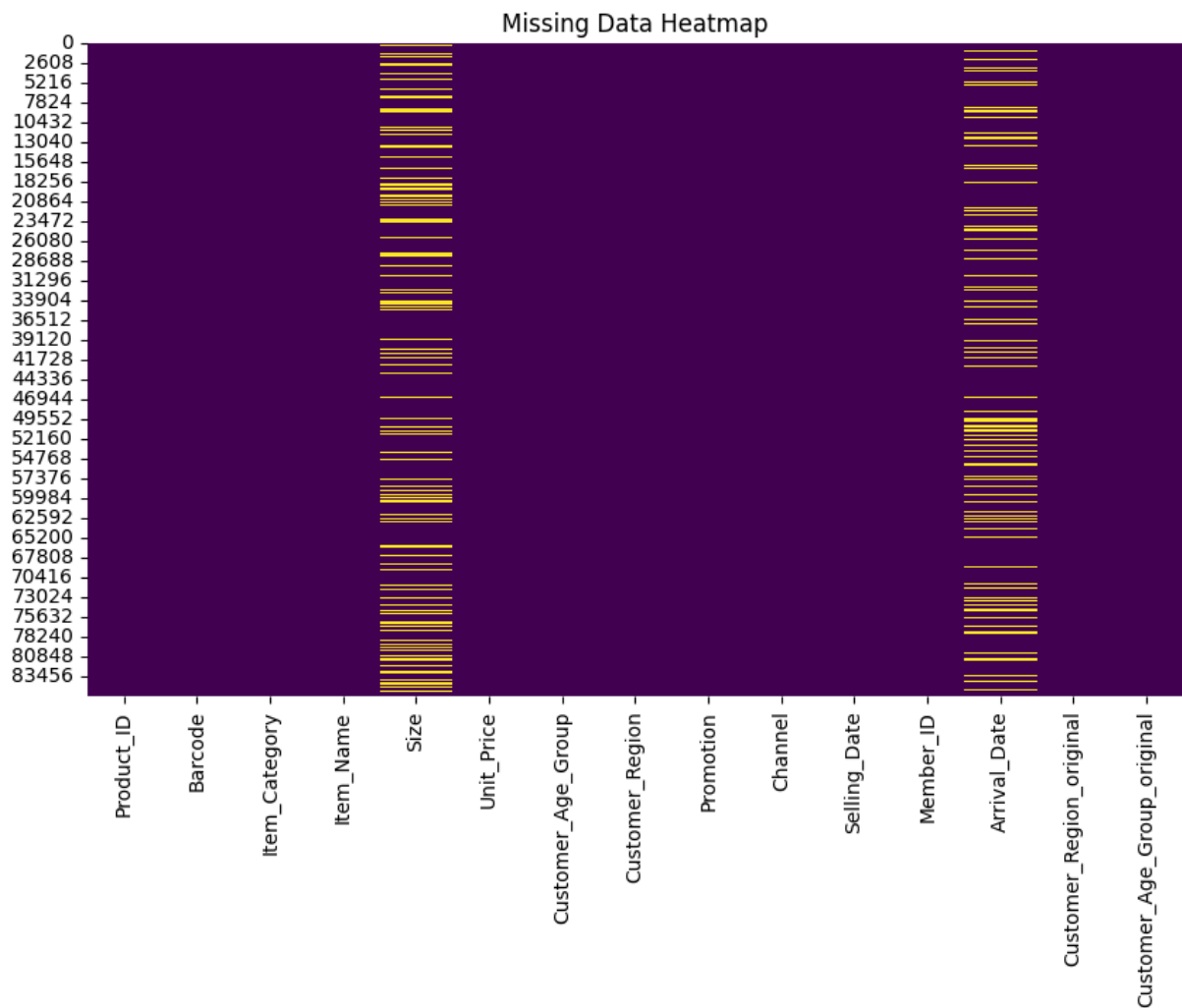
1. Read in cleaned data files

```
In [27]: import pandas as pd
import numpy as np
import altair as alt
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [28]: stadium = pd.read_csv('data/processed/clean_stadium.csv')
fanbase = pd.read_csv('data/processed/fanbase_clean.csv')
merch = pd.read_csv('data/processed/cleaned_merch.csv')
fan_merch_merged = pd.read_csv('data/processed/merch_fanbase_merged.csv')
```

Missing values in Merch

```
In [29]: plt.figure(figsize=(10, 6))
sns.heatmap(merch.isnull(), cbar=False, cmap='viridis')
plt.title("Missing Data Heatmap")
plt.show()
```



Missing At Random (MAR):

I already know from looking at the data that missing values in arrival date align with merch bought in store, and missing sizes align with products that do not require sizes (such as caps, mugs, posters) but this would be good to confirm.

```
In [30]: data = {
    'Product' : merch['Product_ID'].unique(),
    'Item' : merch['Item_Name'].unique()
}

product_table = pd.DataFrame(data)
```

```
In [31]: product_table
```

Out [31]:

	Product	Item
0	10000000	Classic Cap
1	10000001	Retro Cap
2	10000002	Team Hoodie
3	10000003	Training Hoodie
4	10000004	Away Jersey
5	10000005	Home Jersey
6	10000006	Retro Jersey
7	10000007	Club Mug
8	10000008	Logo Mug
9	10000009	Player Poster
10	10000010	Team Poster
11	10000011	Away Scarf
12	10000012	Home Scarf
13	10000013	Youth Team Hoodie
14	10000014	Youth Training Hoodie
15	10000015	Youth Away Jersey
16	10000016	Youth Home Jersey
17	10000017	Youth Retro Jersey

```
In [32]: size_ids = [10000002, 10000003, 10000004, 10000005, 10000006, 10000013, 10000014]
```

```
In [33]: merch['should_have_size'] = merch['Product_ID'].isin(size_ids)

merch['size_missing'] = merch['Size'].isna()
```

```
In [34]: merch['size_missing'] = merch['Size'].isna()

mismatches = merch[merch['should_have_size'] & merch['size_missing']]
```

```
In [35]: mismatches.shape
```

Out [35]: (0, 17)

OK, we confirm that all missing values in the size column are due to products not needing sizes.

```
In [36]: merch = merch.drop(['size_missing', 'should_have_size'], axis = 1)
```

Confirm that all missing values in arrival date are due to products being bought in store and not online:

```
In [37]: merch['arrival_missing'] = merch['Arrival_Date'].isna()

merch['should_have_arrival'] = merch['Channel'] != 'Team Store'

arrival_mismatches = merch[merch['arrival_missing'] & merch['should_have_arrival']]

store_date_mismatches = merch[~merch['arrival_missing'] & (merch['Channel'] == 'Store')]
```

```
In [38]: arrival_mismatches.shape
```

```
Out[38]: (0, 17)
```

```
In [39]: store_date_mismatches.shape
```

```
Out[39]: (0, 17)
```

```
In [40]: merch = merch.drop(['arrival_missing', 'should_have_arrival'], axis = 1)
```

```
In [41]: merch.head()
```

```
Out[41]:
```

	Product_ID	Barcode	Item_Category	Item_Name	Size	Unit_Price	Customer_Ag
0	10000000	20000000	Cap	Classic Cap	NaN	35	
1	10000000	20000001	Cap	Classic Cap	NaN	35	
2	10000000	20000002	Cap	Classic Cap	NaN	35	
3	10000000	20000003	Cap	Classic Cap	NaN	35	
4	10000000	20000004	Cap	Classic Cap	NaN	35	

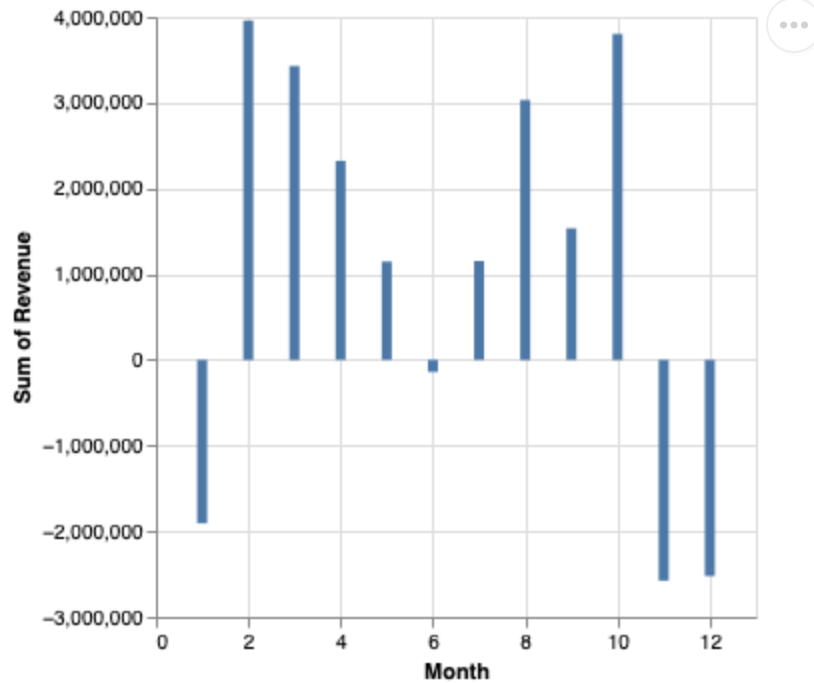
Ok, we have confirmed that all missing values are logical and values that should have input, do have input

EDA: Stadium

```
In [42]: overall_revenue = alt.Chart(stadium).mark_bar().encode(
    x = 'Month',
    y = 'sum(Revenue)',
)
```

overall_revenue

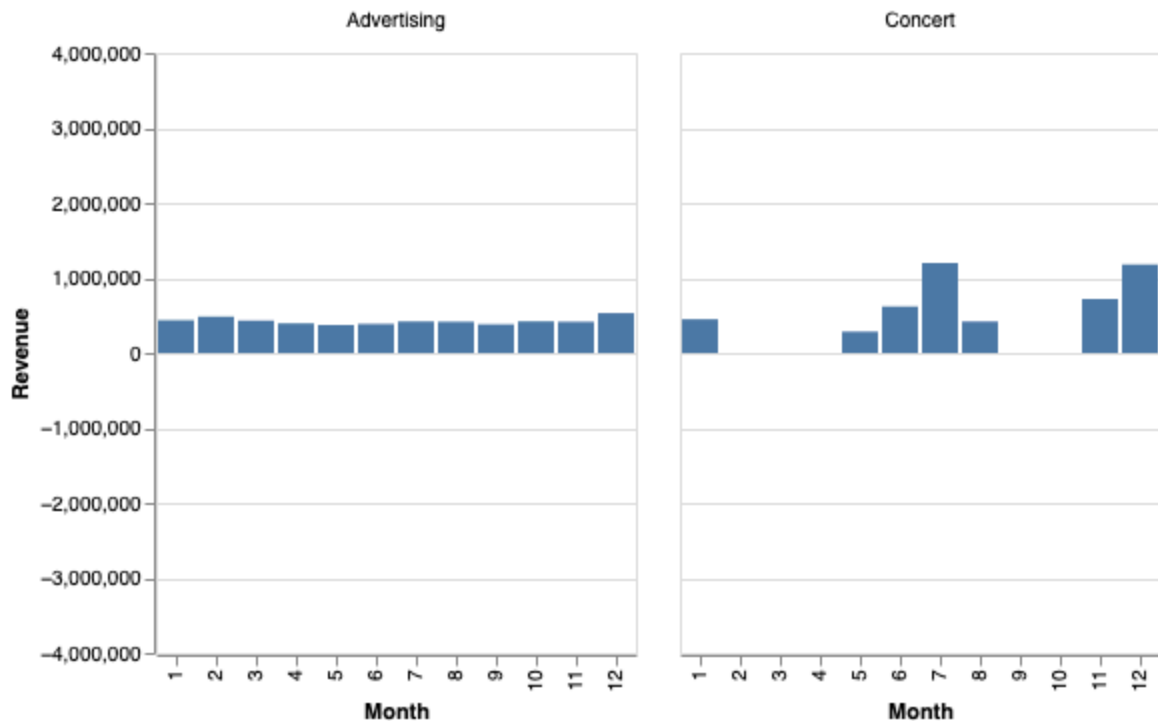
Out [42]:



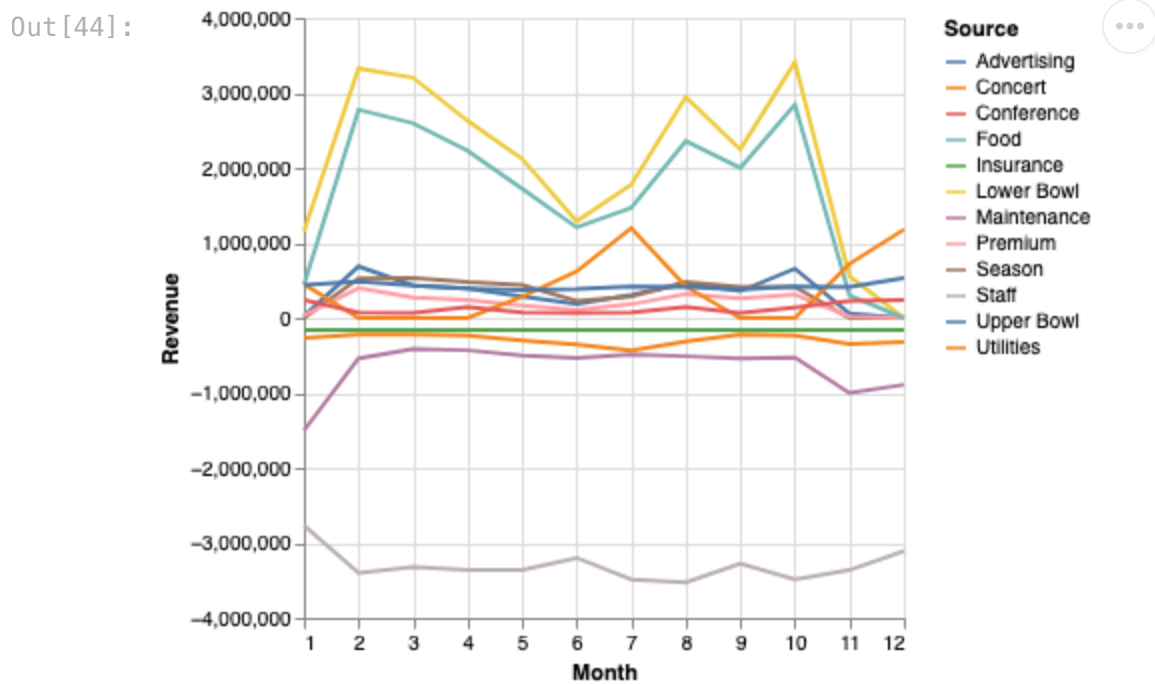
```
In [43]: rev_gain_or_loss = alt.Chart(stadium).mark_bar().encode(
    x='Month:O',
    y='Revenue:Q',
    color='Revenue_Flag:N',
    column='Source:N'
)

rev_gain_or_loss
```

Out [43]:



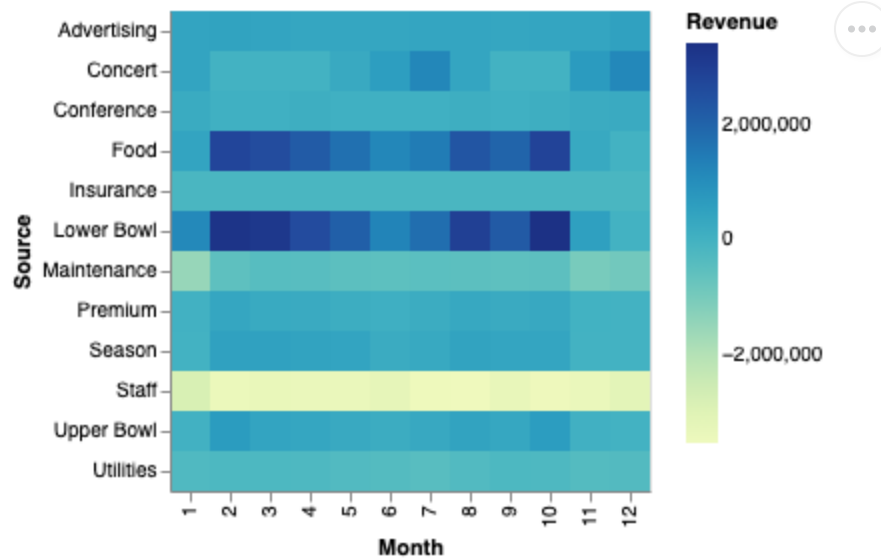
```
In [44]: revenue_by_source = alt.Chart(stadium).mark_line().encode(  
    x = 'Month',  
    y = 'Revenue',  
    color = 'Source'  
)  
  
revenue_by_source
```



```
In [45]: alt.Chart(stadium).mark_rect().encode(  
    x='Month:0',
```

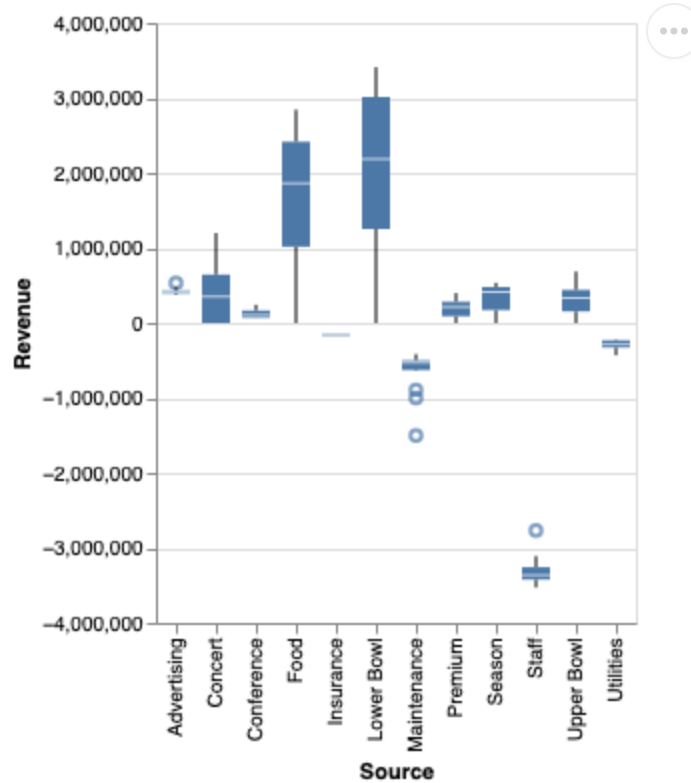
```
y='Source:N',
color='Revenue:Q'
)
```

Out [45]:



```
In [46]: alt.Chart(stadium).mark_boxplot().encode(
x='Source:N',
y='Revenue:Q'
)
```

Out [46]:



Insights:

- primary sources of income are from Lower Bowl and Food
- Upper Bowl is significantly lower earning than LB

- The highest source of loss is Staff
- February and October have the overall highest profit
- January, June, November, December are overall losses

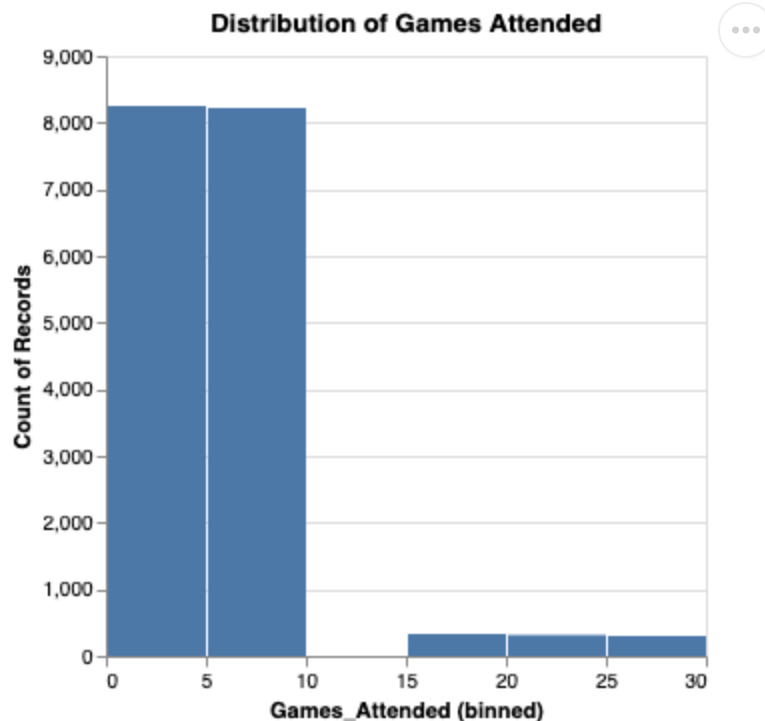
EDA: Fanbase

```
In [47]: alt.data_transformers.disable_max_rows()
```

```
Out[47]: DataTransformerRegistry.enable('default')
```

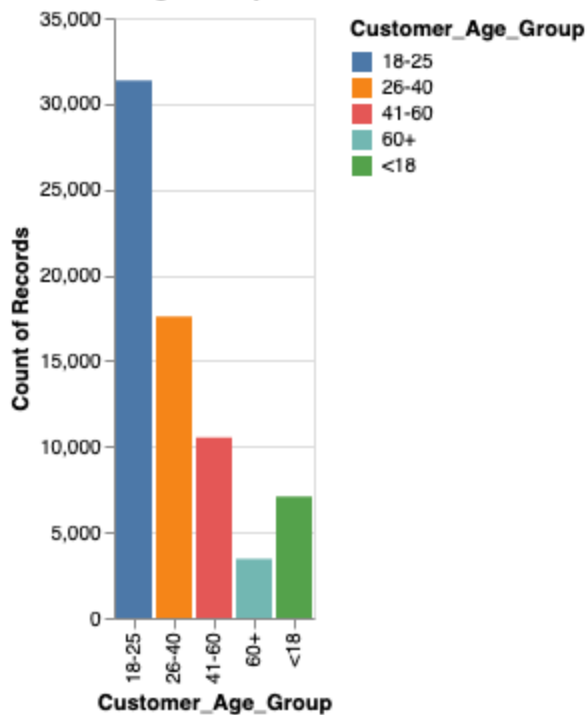
```
In [48]: alt.Chart(fanbase).mark_bar().encode(
    x=alt.X('Games_Attended:Q', bin=True),
    y='count()',
    tooltip=['Games_Attended'])
.properties(title='Distribution of Games Attended')
```

```
Out[48]:
```



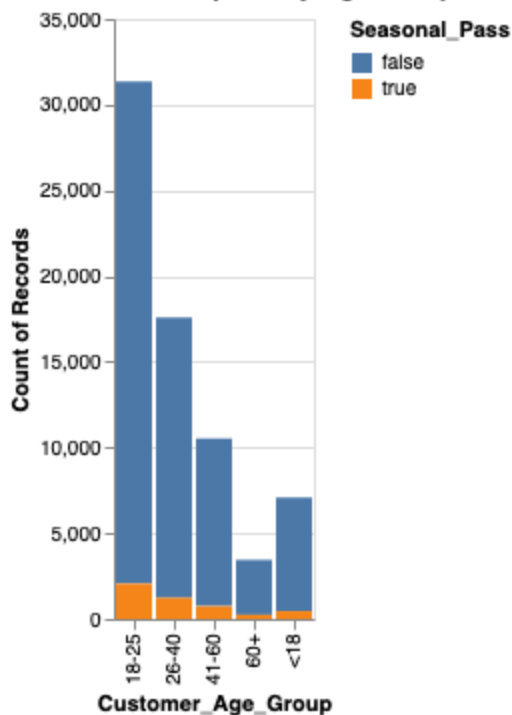
```
In [49]: alt.Chart(fanbase).mark_bar().encode(
    x='Customer_Age_Group:N',
    y='count()',
    color='Customer_Age_Group:N')
.properties(title='Customer Age Group Distribution')
```


Out [49]: **Customer Age Group Distribution**



```
In [50]: alt.Chart(fanbase).mark_bar().encode(  
    x='Customer_Age_Group:N',  
    y='count()',  
    color='Seasonal_Pass:N'  
).properties(title='Seasonal Pass Adoption by Age Group')
```

Out [50]: **Seasonal Pass Adoption by Age Group**



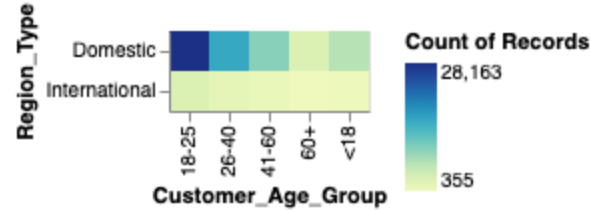
```
In [51]: alt.Chart(fanbase).mark_rect().encode(  
    x='Customer_Age_Group:N',  
    y='Region_Type:N',
```

```

        color='count()',
        tooltip=['Customer_Age_Group', 'Region_Type']
    ).properties(title='Customer Distribution by Region Type and Age Group')

```

Out [51]: **Customer Distribution by Region Type and Age Group** ...

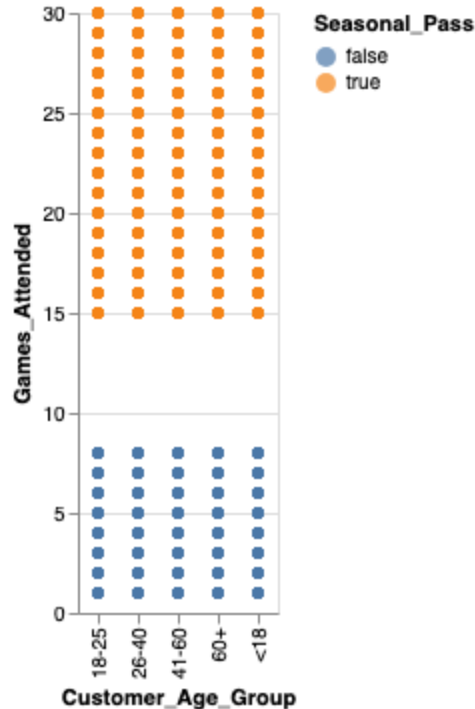


```

In [52]: alt.Chart(fanbase).mark_circle().encode(
        x='Customer_Age_Group:N',
        y='Games_Attended:Q',
        color='Seasonal_Pass:N',
        tooltip=['Member_ID', 'Games_Attended', 'Seasonal_Pass']
    ).properties(title='Games Attended by Age Group and Pass Status')

```

Out [52]: **Games Attended by Age Group and Pass Status** ...

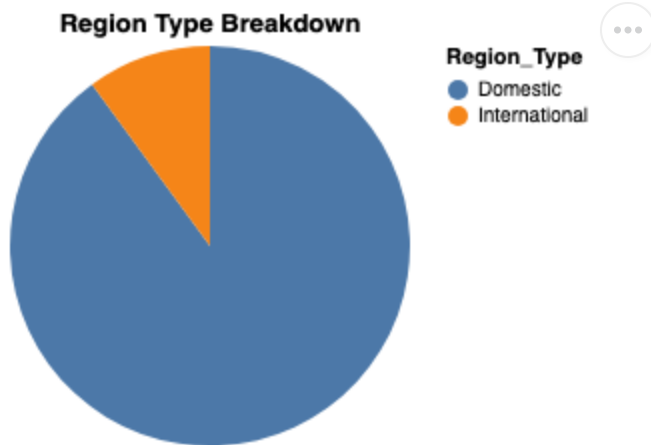


```

In [53]: alt.Chart(fanbase).mark_arc().encode(
        theta='count()',
        color='Region_Type:N',
        tooltip=['Region_Type']
    ).properties(
        title='Region Type Breakdown',
        width=200,
        height=200
    )

```

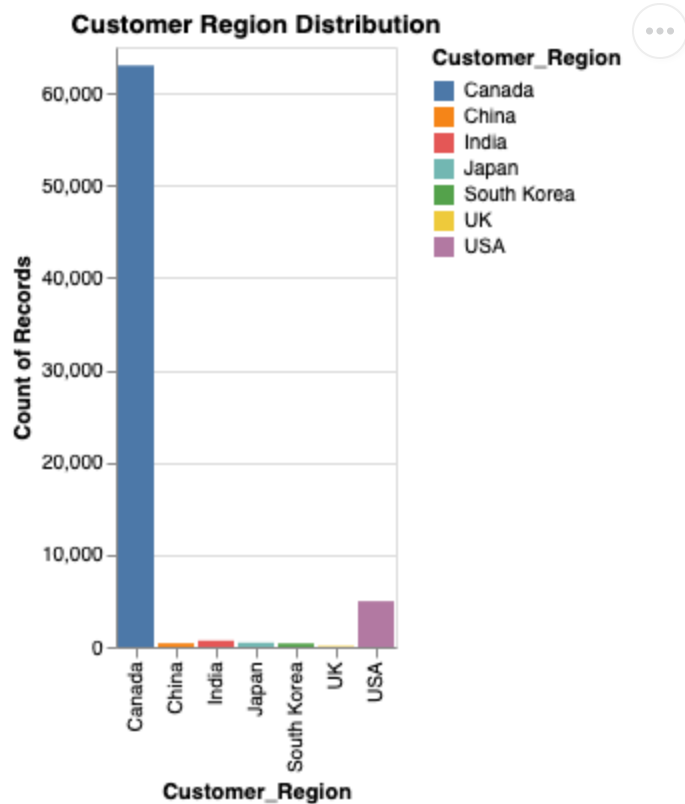
Out [53]:



In [54]:

```
alt.Chart(fanbase).mark_bar().encode(  
    x='Customer_Region:N',  
    y='count()',  
    color='Customer_Region:N'  
) .properties(title='Customer Region Distribution')
```

Out [54]:

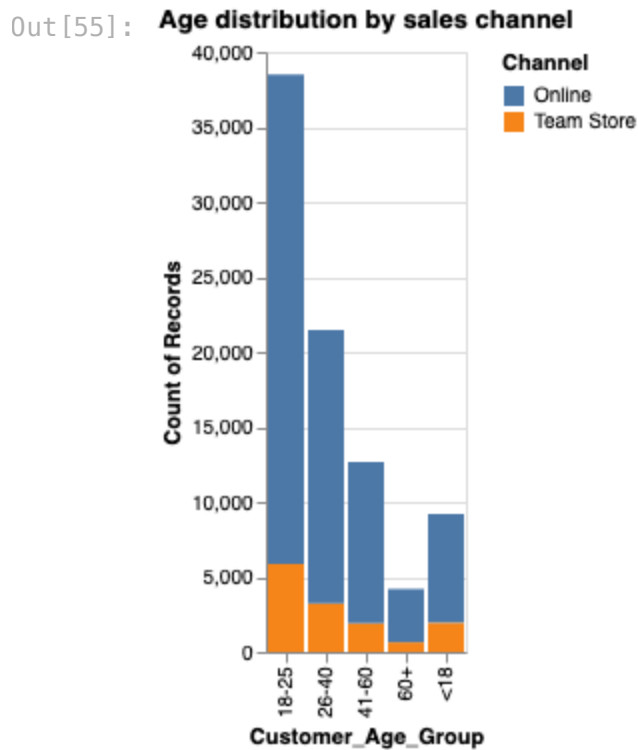


Insights:

- The vast majority of memberships come from Domestic fans
- The largest demographic of fans are 18-25 year olds
- Most people attend 0-10 games
- People who see 15-20 games are more likely to be season pass holders
- Seasonal pass holders make up about 10% of the fanbase
- Of the international fans - the majority come from the US, followed by India

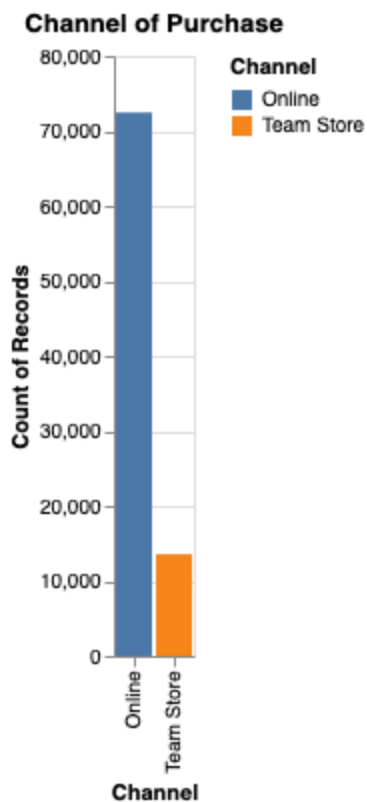
EDA: Merch

```
In [55]: alt.Chart(merch).mark_bar().encode(  
    x='Customer_Age_Group',  
    y='count()',  
    color='Channel:N'  
).properties(title='Age distribution by sales channel')
```



```
In [56]: alt.Chart(merch).mark_bar().encode(  
    x='Channel:N',  
    y='count()',  
    color='Channel:N'  
).properties(title='Channel of Purchase')
```

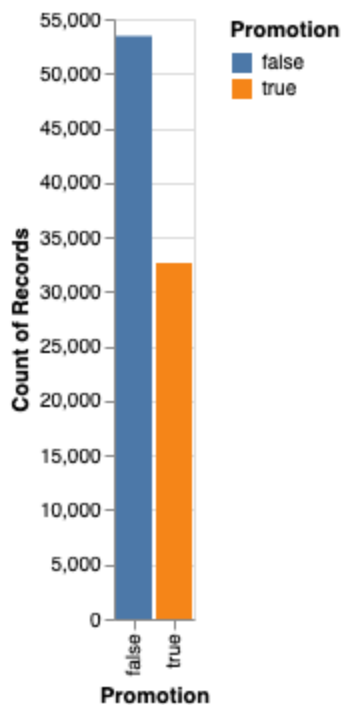
Out [56]:



```
In [57]: alt.Chart(merch).mark_bar().encode(  
    x='Promotion:N',  
    y='count()',  
    color='Promotion:N'  
) .properties(title='Distribution of Promotion Based Purchases')
```

Out [57]:

Distribution of Promotion Based Purchases



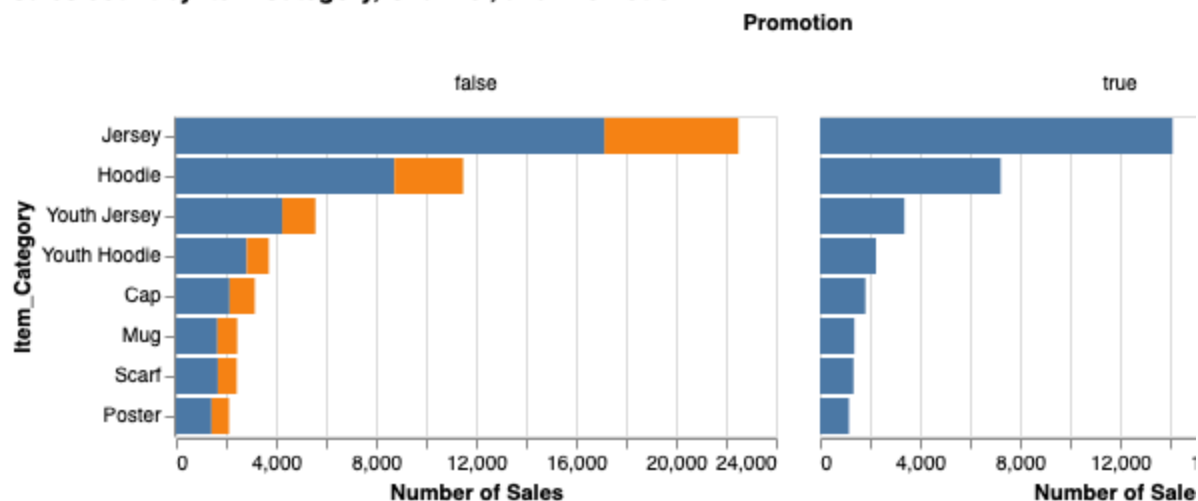
```
In [58]: alt.Chart(merch).mark_bar().encode(  
    x=alt.X('count()', title='Number of Sales'),
```

```

y=alt.Y('Item_Category:N', sort='-x'),
color='Channel',
column=alt.Column('Promotion:N', title='Promotion')
).properties(
    title='Sales count by Item Category, Channel, and Promotion'
)

```

Out [58]: **Sales count by Item Category, Channel, and Promotion**

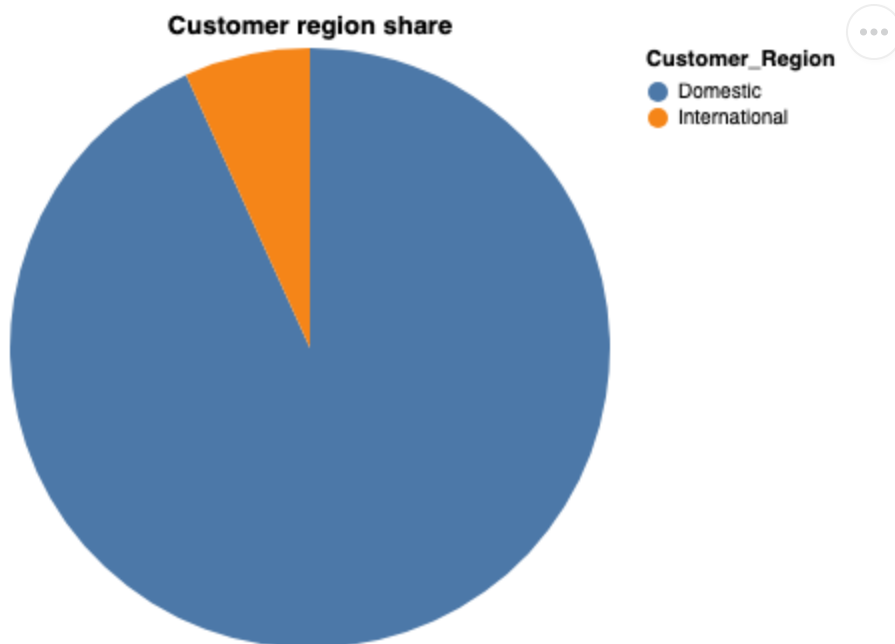


```

In [59]: alt.Chart(merch).mark_arc().encode(
    theta='count()',
    color='Customer_Region:N'
).properties(title='Customer region share')

```

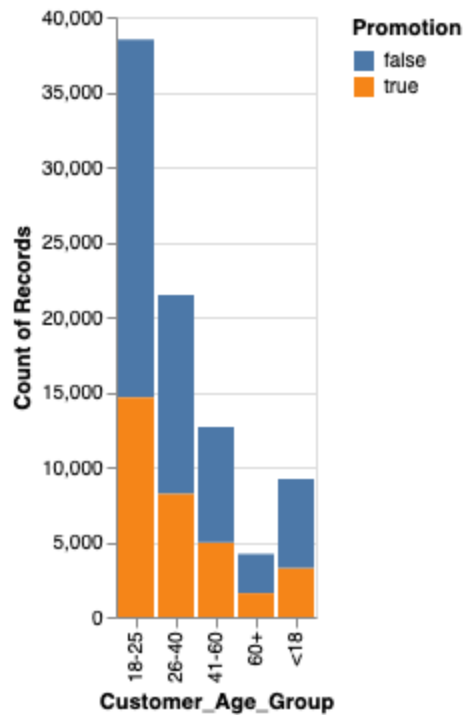
Out [59]:



In [60]: *#fix this, hard to tell actual proportions per age group*

```
alt.Chart(merch).mark_bar().encode(
    x='Customer_Age_Group:N',
    y='count()',
    color='Promotion:N',
).properties(title='Promotional vs non-promotional sales by age group and ch
```

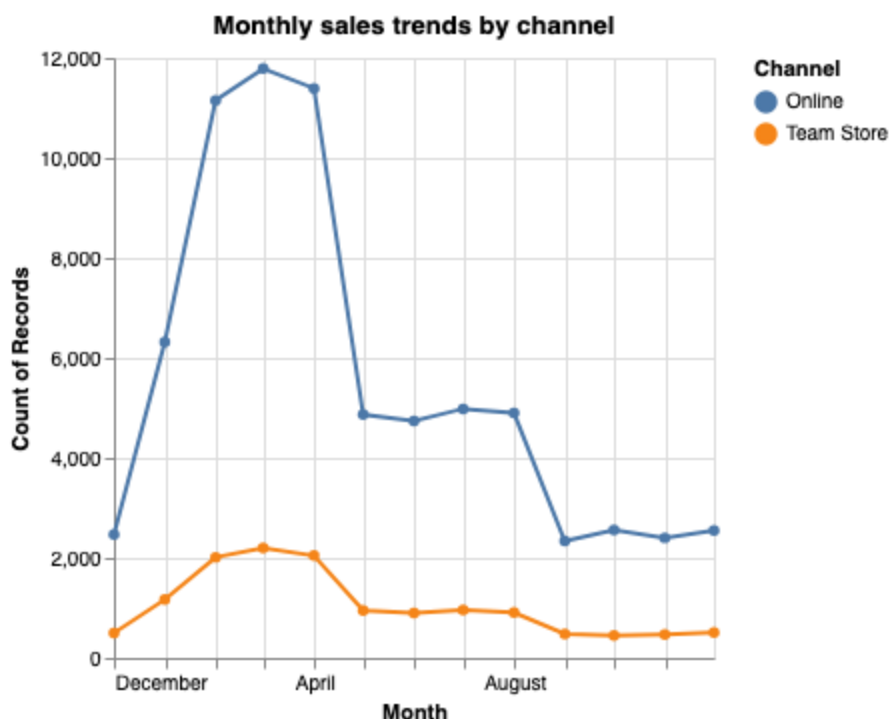
Out [60]: **Promotional vs non-promotional sales by age group and channel** ...



In [61]: `merch['Selling_Date'] = pd.to_datetime(merch['Selling_Date'])`
`merch['Month'] = merch['Selling_Date'].dt.to_period('M').astype(str)`

```
alt.Chart(merch).mark_line(point=True).encode(
    x='Month:T',
    y='count()',
    color='Channel:N'
).properties(title='Monthly sales trends by channel')
```

Out [61]:



```
In [62]: merch['Arrival_Date'] = pd.to_datetime(merch['Arrival_Date'])
merch['Selling_Date'] = pd.to_datetime(merch['Selling_Date'])

merch['delivery_days'] = (merch['Arrival_Date'] - merch['Selling_Date']).dt.

alt.Chart(merch.dropna(subset=['delivery_days'])).mark_bar().encode(
    x=alt.X('delivery_days:Q', bin=alt.Bin(maxbins=20), title='Days to arriv
    y='count()',
    color = 'Customer_Region'
).properties(title='Distribution of delivery times')
```

Out [62]:

Insights:

- 18-25 year olds are the largest demographic for merch purchases, followed by 26-40 year olds*
- the majority of purchases happen online
- majority of purchases do not happen with a promotion, but a reasonably large amount are through a promotion
- all purchases through a promotion happen online (makes sense if promotions are online ads)
- the vast majority of merch purchases occur domestically*
- there does not seem to be a trend between age demographic and use of promotions
- major peak in purchases around March, with a dip in September
- most purchases take around 8 days to be delivered, with some delays leading to 9-10 day delivery

*note these values have been transformed based on the assumptions made in the data cleaning - exact values may not be accurate

To Do: EDA merged merch/fanbase data

- potentially build a model predicting members more likely to purchase merch - can use this to push promotions